OpenHW Studio

Frontend Team — Month 1 Progress Report

Feb-Mar 2026

Interns:

1. Satvik Sharma

2. Md. Danish

3. S. Aaditya Pranav

4. Akshat Singh Tomar

5. Riddhima Singh

6. Kartikay Goel

7. Mohit Sharma

8. Pavan Nitin Nimkar

Group Leader: Satvik Sharma

This document summarises the work completed by the frontend team during the first month of development on OpenHW Studio. It covers what was built, how it works, decisions we made along the way, and what we're planning next.

1. Overview

OpenHW Studio is a web-based electronics education platform. It lets students design circuits, write Arduino code, and run hardware simulations directly in the browser — no physical components needed. Teachers can manage classes, post assignments, and track student progress. Admins can manage the component library and review community submissions, and gamified learning for students.

2. Detailed Feature Breakdown

2.1 Authentication

We built a complete auth system supporting both email/password and Google OAuth. A few things worth noting about the design:

- We maintain two completely separate session tracks — one for students/teachers and one for admins. Each has its own localStorage key so an admin session can't be used on student/teacher routes and vice versa.
- Google OAuth uses a redirect flow. When the user returns from Google, the token in the URL is picked up, the profile is fetched from the backend, and then the URL is cleaned up so it doesn't sit in browser history.
- A ProtectedRoute component wraps all role-sensitive routes. If the user isn't authenticated or doesn't have the right role, they get redirected to sign in.

2.2 Simulation Engine

The simulation engine runs a full 16 MHz AVR CPU emulation in a Web Worker, keeping the UI thread free for rendering at 60 FPS.

How it works

The user writes Arduino C++ code in the editor, clicks Run, and the code gets compiled to a .hex file by the backend. The frontend then sends a START message to the Web Worker along with the .hex, the circuit topology (components + wires), and any custom component logic. The worker loads the hex into the AVR CPU, boots it up, and starts posting pin state updates back to the main thread 60 times per second. The main thread takes those updates and applies them to the Wokwi DOM elements on the canvas.

Component support

The worker has a registry of component types mapped to their simulation logic classes. Out of the box this covers: LED, Resistor, Pushbutton, Potentiometer, Slide Pot, Servo, Buzzer, Motor, Motor Driver, NeoPixel Matrix, Joystick, Shift Register, and a 4x4 Keypad. Custom community components are injected into this registry at runtime when the worker receives a START message — they're transpiled in-browser and eval'd into the registry without any page reload.

2.3 Offline Storage

We built three IndexedDB stores from scratch (no third-party library) to handle offline use cases:

Store	What it stores
hexCache	Compiled .hex results, keyed by a hash of the code + board type. Max 50 entries, LRU eviction. Means you can re-run a simulation offline after compiling at least once.
componentQueue	ZIP component submissions that couldn't reach the backend (offline). Drained automatically when the connection comes back.
projects	Full circuit state — components, wires, code, board type. One record per project, scoped by user email (or "guest" for unauthenticated users). Auto-saves every 2.5 seconds.

Guest users are fully supported. The project store just uses "guest" as the owner key. This was a deliberate choice to lower the barrier to entry for first-time visitors.

We also added a Service Worker for app-shell caching. Navigation requests use a network-first strategy with a fallback to cached index.html. Static assets (JS, CSS, CDN scripts) use stale-while-revalidate. API routes are always network-only — hex caching is handled at the app layer, not in the service worker.

2.4 Classroom LMS

The classroom module gives teachers a space to manage classes and students, and gives students a home for their coursework.

Teacher side

- Create and manage classrooms
- Invite students by email or share a join code
- Post assignments with due dates and point values
- Post notices and announcements to the class stream
- View submissions for each assignment
- Remove students from a class

Student side

- Join a class with an invite code
- View the class stream — assignments and notices in order
- Comment on assignments and notices
- Progressive learning system provided

2.5 Gamification System

We built a full progression system to keep students engaged as they learn. It tracks XP, levels, badges, coins, and completed projects — all stored in localStorage under a per-user key so each account has isolated progress.

There are 10 levels, each requiring progressively more XP. Each level unlocks new projects. For starting a project user needs to unlock required components first, ensuring they learn first.

Each level also comes with a structured project — starter code, wiring instructions, and a set of objectives. When a student submits a project for assessment, we run an auto-evaluation against criteria like: are the right components placed, are the wires correct pin-to-pin, does the code contain the expected patterns, and does the simulation behave as expected.

Badges are awarded for completing levels and hitting milestones. They have rarity tiers (common through legendary) and show up as toast notifications when earned. Same for level-ups and XP awards.

Alongside the gamified flow, each hardware component has a standalone Theory page (with explanations, formulas, and wiring examples) and a Quiz (multiple choice with a passing score threshold), for unlocking the components before starting a project.

2.6 Admin Portal

The admin portal is a three-panel dashboard for managing the platform's component library and Arduino compiler libraries.

- Library Manager — search, install, and uninstall Arduino libraries on the compiler backend
- Pending Submissions — review community-submitted components. Admins can run a live transpile check, download the source ZIP, open the component in an isolated test simulator, then approve or reject. Approval pushes the component to the backend and live-injects it into all active simulator sessions. Rejection removes only that specific submission by its unique ID.
- Installed Components — browse and uninstall approved components. Uninstalling immediately purges the component from all running simulator sessions without a reload.

2.7 Other Pages

- Landing page — public-facing intro to the platform
- Projects Gallery — filterable list of all available guided projects with difficulty, estimated time, and XP
- Project Guide — step-by-step build instructions for each project
- Project Assessment — auto-graded evaluation with a score breakdown
- Theory pages — per-component theory with sections and explanations
- Quiz pages — interactive quiz with scoring and retry
- Blockly Editor — visual block-based programming environment as an alternative to writing code
- Role selection — post-login picker for student vs teacher

2.8 Deliverables Completed in Month 1

Feature / Module	Status	Notes
Authentication System (Email + Google OAuth)	**Complete**	JWT + localStorage, dual session tracks
Role Selection Flow (Student / Teacher / Admin)	**Complete**	Post-login role gate with ProtectedRoute
Sign In / Sign Up Pages	**Complete**	Native credentials + Google OAuth redirect
Landing Page (Public)	**Complete**	Marketing/home page
Student Dashboard	**Complete**	Classroom listing, join by code
Teacher Dashboard	**Complete**	Class creation, management overview
Student Class Detail Page	**Complete**	Assignments, notices, comments
Teacher Class Detail Page	**Complete**	Post assignments, manage students
Circuit Simulator (Main)	**Complete**	Drag-drop canvas, code editor, run/stop
AVR Web Worker Simulation Engine	**Complete**	execute.ts + simulation.worker.ts

Feature / Module	Status	Notes
Multi-board Simulation Mode	**Complete**	UART routing between multiple Arduinos
NeoPixel Matrix Support	**Complete**	setPixel() DOM calls at 60 FPS
Analog Plotter / Logic Graph	**Complete**	Canvas-based pin signal tracer
Serial Monitor (Bidirectional)	**Complete**	AVRUSART streaming in/out
Physical Reset Button	**Complete**	cpu.reset() wired to SVG button
Project Persistence (IndexedDB)	**Complete**	Auto-save every 2.5s, guest support
Compiled Hex Cache (IndexedDB)	**Complete**	Survives page refresh, 50-entry LRU
Offline ZIP Component Queue	**Complete**	Drain-on-reconnect pattern
Service Worker (App Shell)	**Complete**	Network-first nav, SWR assets
Custom Component Upload (ZIP)	**Complete**	Babel transpile + live injection
Zero-Touch Component Sync (12s poll)	**Complete**	Live add/remove without page reload
Admin Portal (3-panel dashboard)	**Complete**	Libraries, pending, installed
Admin Component Review Workflow	**Complete**	Transpile, test, approve, reject
Gamification System (XP / Levels / Badges)	**Complete**	10 levels, per-user localStorage
Gamification Toast Notifications	**Complete**	Level-up, badge, XP popups
Components Page (Theory + Quiz)	**Complete**	10+ components, quiz scoring, for unlocking components
Projects Gallery	**Complete**	Filterable project browser
Project Guide Page	**Complete**	Step-by-step build instructions
Project Assessment Page	**Complete**	Auto-graded against eval criteria
Gamification Simulator Mode	**Complete**	Pre-loaded project simulator
Blockly Visual Editor	**Complete**	Drag-and-drop block programming, with import and export feature